

torch.float_power#

https://pytorch.org/docs/stable/generated/torch.float_power.html#torch.float

Description:

Raises input to the power of exponent, elementwise, in double precision. If neither input is complex returns a torch.float64 tensor, and if one or more inputs is complex returns a torch.complex128 tensor. This function always computes in double precision, unlike torch.pow(), which implements more typical type promotion. This is useful when the computation needs to be performed in a wider or more precise dtype, or the results of the computation may contain fractional values not representable in the input dtypes, like when an integer base is raised to a negative integer exponent. input (Tensor or Number) – the base value(s) exponent (Tensor or Number) – the exponent value(s)

Function Signature

```
torch.float_power(input, exponent, *, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> a = torch.randint(10, (4,))
>>> a
tensor([6, 4, 7, 1])
>>> torch.float_power(a, 2)
tensor([36., 16., 49., 1.], dtype=torch.float64)

>>> a = torch.arange(1, 5)
>>> a
tensor([ 1,  2,  3,  4])
>>> exp = torch.tensor([2, -3, 4, -5])
>>> exp
tensor([ 2, -3,  4, -5])
>>> torch.float_power(a, exp)
tensor([1.0000e+00, 1.2500e-01, 8.1000e+01, 9.7656e-04],
dtype=torch.float64)
```

Notes & Warnings

NOTE

Note This function always computes in double precision, unlike `torch.pow()`, which implements more typical type promotion. This is useful when the computation needs to be performed in a wider or more precise dtype, or the results of the computation may contain fractional values not representable in the input dtypes, like when an integer base is raised to a negative integer exponent.

Detailed Documentation

Documentation

Raises input to the power of exponent, elementwise, in double precision. If neither input is complex returns a `torch.float64` tensor, and if one or more inputs is complex returns a `torch.complex128` tensor.

This function always computes in double precision, unlike `torch.pow()`, which implements more typical type promotion. This is useful when the computation needs to be performed in a wider or more precise dtype, or the results of the computation may contain fractional values not representable in the input dtypes, like when an integer base is raised to a negative integer exponent.

input (Tensor or Number) – the base value(s)

exponent (Tensor or Number) – the exponent value(s)

out (Tensor, optional) – the output tensor.